

# Neurax Compiler — Zero to Hero Architecture Guide

A complete technical reference covering the Neurax analytic compiler and MLIR code-generation backend, from first principles to full system design.

---

## Table of Contents

1. What Is Neurax?
  2. Motivation & Problem Space
  3. Workspace Structure
  4. The Input Format — Model JSON
  5. Layer 1 — `neurax-parser`
  6. Layer 2 — `neurax-ir` — The 10-Pass Analytic Pipeline
    - Shared Infrastructure
    - Pass 1 — Architecture IR
    - Pass 2 — Graph IR
    - Pass 3 — Tensor IR
    - Pass 4 — Operator IR
    - Pass 5 — Compute IR
    - Pass 6 — Memory IR
    - Pass 7 — Parallelism IR
    - Pass 8 — Hardware IR
    - Pass 9 — Cost IR
    - Pass 10 — Report IR
  7. Layer 3 — `neurax-core` — Pipeline Orchestrator
  8. Supporting Libraries
    - `neurax-formulas`
    - `neurax-hardware-db`
  9. User Interfaces
    - `neurax-cli`
    - `neurax-tui`
  10. Layer 4 — `neurax-mlir` — The Code Generation Backend
  11. Full End-to-End Data Flow
  12. Diagnostic System
  13. Comparisons with Peer Tools
  14. Quality Assessment
  15. Gap Analysis & Roadmap
  16. The Full Vision
-

## 1. What Is Neurax?

**Neurax** is a **Universal Analytic Compiler for AI Architectures** written in Rust. It takes a JSON description of a neural network and statically computes — without ever executing the model — a comprehensive set of metrics covering:

- Parameter counts and architecture statistics
- FLOPs (forward, backward, total per step)
- Memory footprint (parameters, activations, gradients, optimizer state, peak VRAM)
- Tensor shapes and data flow
- Parallelism strategy recommendations
- Hardware performance via the Roofline model
- Training cost in USD and CO<sub>2</sub>

It is both: 1. An **analytic compiler** — transforms a declarative model spec into quantified performance metrics via 10 IR passes 2. A **code generator** (in progress) — lowers those IR structures into MLIR operations that can be compiled to native binary code for any hardware target

---

## 2. Motivation & Problem Space

Training or deploying a large model without upfront analysis leads to:

- Out-of-memory crashes discovered only after hours of setup
- Suboptimal parallelism that wastes 30-60% of GPU utilization
- Unexpected training costs in the hundreds of thousands of dollars
- No systematic way to compare model architectures before committing to a run

Existing tools are either **too narrow** (only transformers, or only memory, or only one GPU vendor) or **require running the model** first (making them useless for planning).

Neurax's design goal: **predict everything before a single GPU is provisioned**, for any model type, on any hardware.

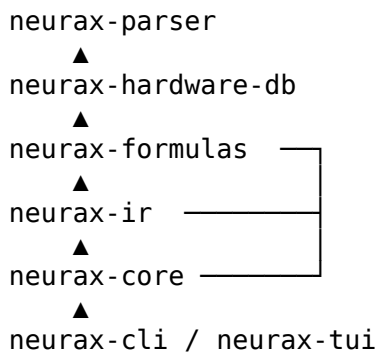
---

## 3. Workspace Structure

|                   |                                         |
|-------------------|-----------------------------------------|
| Neurax-IR/        | ← Cargo workspace root                  |
| ├─ neurax-parser/ | ← JSON ingestion & strong-typed config  |
| ├─ neurax-ir/     | ← 10 IR dialects (core analysis engine) |
| ├─ neurax-core/   | ← Pipeline orchestrator                 |

|                       |                                       |
|-----------------------|---------------------------------------|
| — neurax-formulas/    | ← Per-architecture FLOPs formulas     |
| — neurax-hardware-db/ | ← GPU/CPU/interconnect spec database  |
| — neurax-cli/         | ← Binary: `neurax` command-           |
| line tool             |                                       |
| — neurax-tui/         | ← Binary: Ratatui terminal UI         |
| — neurax-mlir/        | ← MLIR dialect definitions & code gen |

### Dependency graph:



neurax-mlir (independent, bridges to neurax-ir outputs via integration.rs)

## 4. The Input Format — Model JSON

Users describe their model in a structured JSON file. Here is a minimal transformer example:

```

{
  "schema_version": "1.0",
  "model": {
    "name": "MyLLM-7B",
    "type": "transformer",
    "global_params": {
      "hidden_size": 4096,
      "num_attention_heads": 32,
      "num_key_value_heads": 8,
      "intermediate_size": 14336,
      "num_layers": 32,
      "vocab_size": 128000
    },
    "layers": [
      { "id": "embed", "layer_type": "embedding", "params": { "vocab_size": 128000 } },
      { "id": "attn_0", "layer_type": "attention", "params": { "hidden_size": 4096 } },
      { "id": "mlp_0", "layer_type": "mlp", "params": { "hidden_size": 4096, "intermediate_size": 14336 } }
    ]
  }
}

```

```

    ],
  },
  "training": {
    "batch_size": 4,
    "precision": "bf16",
    "gradient_checkpointing": false,
    "zero_stage": 2,
    "parallelism": { "data_parallel": 8, "tensor_parallel": 4, "pipeline_parallel": 1 },
  },
  "hardware": {
    "gpus": [{ "name": "H100-SXM", "count": 64, "memory_gb": 80 }],
    "interconnect": "NVLink4"
  },
  "cost_config": {
    "provider": "AWS",
    "gpu_hour_usd": 4.10,
    "energy_kwh_usd": 0.12
  }
}

```

**Supported model types:** transformer, cnn, moe, diffusion, gnn, rnn, ssm (Mamba/S4/H3/RWKV), gan

**Supported layer types (50+):** attention, mlp, conv, embedding, normalization, pooling, moe, residual\_block, mbconv, inception, dense\_block, convnext\_block, shuffle\_unit, c2f, detection, mamba\_block, s4\_block, h3\_block, rwkv\_block, retention\_block, generator\_block, discriminator\_block, adain, style\_mod, lstm\_block, gru\_block, rnn\_cell, bidirectional, encoder\_block, decoder\_block, custom, and more.

## 5. Layer 1 — neurax-parser

**Purpose:** Parse raw JSON → strongly-typed Rust structures with full validation.

### Key Structs

| Struct      | Description                                                             |
|-------------|-------------------------------------------------------------------------|
| ModelConfig | Root: Model + TrainingConfig + HardwareConfig + DataConfig + CostConfig |

| Struct          | Description                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------------------|
| Model           | ModelType + Vec<Layer> + GlobalParams                                                                                        |
| Layer           | id + LayerType + input_shape + output_shape + LayerParams + Option<CustomEquations>                                          |
| LayerParams     | ~50 fields covering all architectures: attention heads, conv kernels, SSM state dims, GAN style dims, LSTM cell params, etc. |
| CustomEquations | User-defined formulas: flops_forward, memory_activation, gradient (string expressions evaluated by evalexpr)                 |
| TrainingConfig  | Batch size, optimizer, precision, ZeRO stage, ParallelismConfig (DP×TP×PP)                                                   |
| HardwareConfig  | Vec<GpuConfig> — name, count, VRAM, TFLOPS per precision, NVLink, bandwidth                                                  |
| CostConfig      | Provider, GPU-hour USD, energy kWh USD, PUE factor                                                                           |

### Validation Pipeline

```

Raw JSON bytes
  | serde_json deserialization
  ▼
RawModelConfig (unvalidated, all fields optional/string)
  | schema.rs — field presence, type coercions
  ▼
schema_validator.rs — cross-field validation
  | coherence.rs — shape consistency, parameter ranges
  ▼
ModelConfig (fully validated, strongly typed)

```

The parser rejects invalid configs **before** any expensive analysis begins.

## 6. Layer 2 — neurax-ir — The 10-Pass Analytic Pipeline

This is the heart of Neurax. The crate defines **10 IR dialects** in a strict sequential pipeline. Each dialect is a Rust module with 4 files:

- `ir.rs` — data structures for this dialect's output
- `pass.rs` — computation logic implementing `IrPass`
- `metrics.rs` — typed metrics structs
- `mod.rs` — re-exports

### Shared Infrastructure

**NeuraxContext** Created once before any pass runs:

```
pub struct NeuraxContext {  
    pub config: Arc<ModelConfig>,           // Shared parsed config  
    pub gpu_db: Arc<HardwareDatabase>,      // Shared GPU spec database  
    pub compute_config: ComputeConfig,      // Thread count, timeout  
    pub diagnostics: Arc<Mutex<Vec<Diagnostic>>>, // Thread-safe collector  
    metrics_store: Arc<Mutex<HashMap<String, f64>>>, // Inter-pass messaging  
}
```

**IrPass Trait** Every dialect implements this contract:

```
trait IrPass {  
    type Input;  
    type Output;  
    type Metrics;  
  
    fn build(&self, input: &Self::Input, ctx: &NeuraxContext) -> Result<Output>;  
    fn compute_metrics(&self, output: &mut Output, ctx: &NeuraxContext) -> Result<Metrics>;  
    fn validate(&self, output: &Output, metrics: &Metrics) -> Result<()>;  
}
```

---

### Pass 1 — Architecture IR

**Input:** `ModelConfig` **Output:** `ArchitectureIR`

`ModelConfig`

- ├ Enumerate all layers → `Vec<LayerDef>`
- ├ Calculate `param_count` per layer
- ├ Scale to full model size via `global_params.num_layers`
- ├ Detect MoE, GQA, special architectures
- └ Compute `total_parameters`

### Key Metrics (M1-M2):

| Metric           | Description                                |
|------------------|--------------------------------------------|
| total_parameters | Sum across all layers × model scale factor |
| num_layers       | Effective full model depth                 |
| params_per_layer | HashMap<layer_id, param_count>             |
| layers_by_type   | Census of layer types                      |
| model_type_info  | String description                         |

**Special handling:** When the JSON contains only a *sample* of layers (e.g., one transformer block representing 32 identical ones), `global_params.num_layers` is used to scale correctly.

### Pass 2 — Graph IR

**Input:** ArchitectureIR **Output:** GraphIR — a `petgraph::DiGraph<GraphNode, GraphEdge>`

ArchitectureIR

- Create a GraphNode per layer
- Connect with GraphEdge (carrying tensor shape + dtype + size\_bytes)
- Detect and reject cycles (diagnostic E005)
- Compute topological order (Kahn's algorithm)
- DP longest-path → graph\_depth (critical path)

### Key Metrics (M3-M5):

| Metric               | Description                                           |
|----------------------|-------------------------------------------------------|
| graph_depth          | Longest path from input to output                     |
| total_operations     | Number of nodes                                       |
| critical_path_length | Minimum sequential depth                              |
| parallel_paths       | Detected branches (residual connections, MoE routing) |
| edge_count           | Total data dependencies                               |

### Pass 3 — Tensor IR

**Input:** GraphIR **Output:** TensorIR

Maps every edge in the graph to a named TensorInfo with shape, dtype, size in bytes, producer layer, and consumer layers. The Shape type supports three dimension kinds:

```
enum Dim {
    Known(usize),           // Concrete: e.g. 4096
    Symbolic(String),       // Named: e.g. "batch", "seq"
    Dynamic,                // Unknown at compile time
}
```

### Key Metrics (M9-M12):

| Metric                   | Description                             |
|--------------------------|-----------------------------------------|
| activation_memory_bytes  | Sum of all intermediate tensor sizes    |
| resolution_ratio         | Fraction of dims that are fully Known   |
| unresolved_dim_count     | Symbolic/Dynamic dims remaining         |
| largest_tensor_id        | Heaviest single tensor                  |
| tensor_size_distribution | Histogram: tiny/small/medium/large/huge |

A `resolution_ratio < 1.0` triggers diagnostic W002 (symbolic dimensions remaining) which lowers the analysis confidence score.

## Pass 4 — Operator IR

**Input:** (TensorIR, ArchitectureIR) **Output:** OperatorIR

This pass **decomposes every layer into atomic operations (AtomOp)**. It is the bridge between high-level layers and actual computation counts.

### 24+ AtomOp types:

Core: MatMul, BatchedMatMul, Conv2D, DepthwiseConv2D, Linear  
 Attention: Attention, FlashAttention, GroupedQueryAttention, MultiQueryAttention, AttentionScores, AttentionOutput, Softmax  
 Normalization: LayerNorm, BatchNorm, RMSNorm, GroupNorm  
 Embeddings: Embedding, TokenEmbedding, PositionalEmbedding, RotaryEmbedding  
 MoE: MoE, MoERouter, MoEE expertGroup  
 SSM/Mamba: SsmStateUpdate, MambaConv1d, S4Block, H3Block  
 RNN: LstmCell, GruCell, RnnCell  
 Fine-tuning: LoRALinear  
 Activations: Add, Mul, Div, ReLU, GELU, SiLU, Tanh, Sigmoid  
 Pooling: Pooling(Max/Avg)  
 Tensor ops: Reshape, Transpose, Concat, Split  
 Custom: Custom (evaluated via `neurax-formulas::custom`)

Each AtomOp carries:

```
pub struct AtomOp {  
    pub op_type: OpType,  
    pub flops: f64,           // Analytically computed  
    pub param_count: u64,  
    pub activation_memory: u64,  
    pub is_custom: bool,  
}
```

The per-architecture FLOPs formulas come from neurax-formulas.

---

## Pass 5 — Compute IR

**Input:** OperatorIR **Output:** ComputeIR

Aggregates all AtomOp.flops into the full compute picture.

### Key Metrics (M13-M18):

| Metric               | Description                                                        |
|----------------------|--------------------------------------------------------------------|
| forward_flops        | Sum of all AtomOp forward FLOPs                                    |
| backward_flops       | $\approx 2 \times$ forward (per-op backward formula in roadmap)    |
| optimizer_flops      | Adam/AdamW momentum + variance updates                             |
| total_step_flops     | forward + backward + optimizer                                     |
| flops_per_token      | For autoregressive models                                          |
| arithmetic_intensity | FLOPs / bytes_accessed — key roofline input                        |
| complexity_class     | $O(n)$ , $O(n \log n)$ , $O(n^2)$ , $O(n^3)$ — attention detection |
| macs                 | $\text{total\_flops} / 2$                                          |

The complexity\_class is determined by the dominant OpType:  
- Standard Attention  $\rightarrow O(n^2)$  (quadratic in sequence length) -  
FlashAttention  $\rightarrow$  upgraded to  $O(n \log n)$  - Conv2D  $\rightarrow O(n)$  (linear in spatial dimensions)

---

## Pass 6 — Memory IR

**Input:** (ComputeIR, TensorIR, ArchitectureIR) **Output:** MemoryIR

The most complex pass. Simulates the full memory lifecycle of a training step.

### Memory components:

Peak VRAM = Parameters + Activations + Gradients + Optimizer State  
+ Intra-  
op buffers (roadmap)

| Component       | Formula                                                   |
|-----------------|-----------------------------------------------------------|
| Parameters      | <code>total_params × bytes_per_element</code>             |
| Activations     | Tensor liveness simulation across the graph               |
| Gradients       | = Parameters (same size as weights)                       |
| Optimizer State | Adam: $2 \times$ Parameters (momentum + variance); SGD: 0 |

The pass also constructs a **memory timeline**: at each execution step, which tensors are live and what is the total allocation. This enables pinpointing the exact operation that causes peak VRAM.

#### OOM Risk classification:

peak / VRAM\_available:  
 < 80% → Safe  
 80–95% → Warning (W005 diagnostic)  
 95–99% → Critical  
 > 100% → Overflow (E001 diagnostic – blocks analysis)

#### Key Metrics (M19-M25):

| Metric                               | Description                          |
|--------------------------------------|--------------------------------------|
| <code>peak_vram_bytes</code>         | Maximum concurrent memory usage      |
| <code>parameter_memory_bytes</code>  | Weights only                         |
| <code>activation_memory_bytes</code> | Intermediate feature maps            |
| <code>gradient_memory_bytes</code>   | Backprop gradients                   |
| <code>optimizer_state_bytes</code>   | Optimizer accumulators               |
| <code>max_batch_size_fit</code>      | Largest batch that fits without OOM  |
| <code>oom_risk</code>                | Safe / Warning / Critical / Overflow |

### Pass 7 — Parallelism IR

**Input:** (MemoryIR, GraphIR) **Output:** ParallelismIR

Evaluates six parallelism strategies and selects the optimal one:

| Strategy                                                 | When optimal                                        |
|----------------------------------------------------------|-----------------------------------------------------|
| DataParallel { num_gpus, efficiency }                    | Model fits on 1 GPU; scale batch size               |
| TensorParallel { tp_degree }                             | Large matmuls that split cleanly across GPUs        |
| PipelineParallel { stages, micro_batches, bubble_ratio } | Very deep models, bandwidth-limited                 |
| Hybrid { dp, tp, pp }                                    | Large models requiring 3D parallelism (GPT-4 style) |
| ZeRO { stage, memory_per_gpu }                           | ZeRO-1/2/3 for memory efficiency in DP              |
| ModelParallel { splits }                                 | Custom layer-range partitioning                     |

### Key Metrics (M26-M30):

| Metric                   | Description                              |
|--------------------------|------------------------------------------|
| data_parallel_efficiency | Linear scaling ratio (Amdahl-adjusted)   |
| communication_overhead   | Fraction of step time spent on AllReduce |
| optimal_gpu_count        | Recommended minimum GPU count            |
| pipeline_stages          | Optimal PP stages                        |
| allreduce_time_ms        | Time for gradient synchronization        |
| scaling_efficiency_curve | Vec<(num_gpus, efficiency)>              |

### Pass 8 — Hardware IR

**Input:** (ComputeIR, MemoryIR, ParallelismIR) **Output:** HardwareIR

Implements the **Industrial Roofline Model** — a 4-level hierarchy:

| Level            | What it models                                     |
|------------------|----------------------------------------------------|
| Classic          | Peak TFLOPS vs. Peak HBM bandwidth                 |
| Calibrated       | Efficiency per op type (attention ≠ matmul ≠ conv) |
| Memory Hierarchy | L1 SRAM / L2 / HBM separately                      |
| Industrial       | Compute-memory overlap, kernel launch overhead     |

The GPU profile is resolved from neurax-hardware-db using the name in the JSON:

```

GpuProfile {
    tflops_fp32, tflops_fp16, tflops_bf16, tflops_int8, tflops_fp8,
    hbm_bandwidth_gb_s, l2_bandwidth_tb_s, sram_bandwidth_tb_s,
    l2_cache_mb, sram_per_sm_kb, num_sms,
    nvlink_bandwidth_gb_s, tdp_watts, ...
}

```

Per-layer LatencyEstimate is computed:

```

t_compute = flops / (peak_tflops × efficiency)
t_memory  = bytes / bandwidth[cache_level]
t_effective = max(t_compute, t_memory) + overlap_reduction
t_total    = t_effective + kernel_launch_overhead

```

### Key Metrics (M31-M35):

| Metric                  | Description                                         |
|-------------------------|-----------------------------------------------------|
| latency_ms              | Estimated time per training step                    |
| throughput_tokens_per_s | Tokens processed per second                         |
| gpu_utilization         | Effective TFLOPS / peak TFLOPS                      |
| bottleneck              | compute-bound / memory-bound / balanced             |
| roofline_position       | 0.0 = fully memory-bound, 1.0 = fully compute-bound |

**Note:** Passes 7 and 8 run **in parallel** using `rayon::join()` since they are independent given passes 1-6. Hardware is then re-run once with the actual parallelism data for final accuracy.

## Pass 9 — Cost IR

**Input:** (HardwareIR, ParallelismIR) **Output:** CostIR

Translates hardware metrics into real-world financial and environmental costs.

### Formulas:

```

training_time_hours = (total_steps × latency_ms) / 3_600_000
gpu_hours_total     = training_time_hours × total_gpus
training_cost_usd   = gpu_hours_total × gpu_hour_usd
energy_kwh          = (total_gpus × tdp_watts × training_time_hours × pue) / 1000
co2_kg              = energy_kwh × co2_per_kwh (0.233 kg/kWh EU average)
cost_per_token      = training_cost_usd / (total_steps × batch_size × seq_len)

```

### Key Metrics (M36-M40):

| Metric                      | Description                  |
|-----------------------------|------------------------------|
| training_cost_usd           | Total GPU rental cost        |
| training_time_hours         | Wall-clock training duration |
| energy_kwh                  | Total electricity consumed   |
| co2_kg                      | Carbon footprint             |
| cost_per_million_tokens_usd | Inference economics          |
| monthly_inference_cost_usd  | Sustained inference budget   |

---

## Pass 10 — Report IR

**Input:** All 9 previous IRs **Output:** ReportIR

The final consolidation pass. Assembles all 43 metrics into a single AllMetrics struct, collects all diagnostics fired during previous passes, generates Recommendation objects, and computes a confidence\_score.

**Confidence score** degrades with: - Unresolved symbolic dimensions (resolution\_ratio < 1.0) - Custom layers without formulas - Missing global params that required estimation

Output formats: - **Markdown** — human-readable report with tables and sections - **JSON** — machine-parsable with full metric tree

---

## 7. Layer 3 — neurax-core — Pipeline Orchestrator

neurax-core exposes the single public API:

```
pub fn run_analysis(config: ModelConfig) -> Result<AnalysisResult, NeuraxError>
```

Internally it sequences all 10 IrPass executions, handling the rayon::join() for passes 7+8, then wraps everything in:

```
pub struct AnalysisResult {
    pub arch: ArchitectureIR,
    pub graph: GraphIR,
    pub tensor: TensorIR,
    pub operator: OperatorIR,
    pub compute: ComputeIR,
    pub memory: MemoryIR,
    pub parallelism: ParallelismIR,
    pub hardware: HardwareIR,
    pub cost: CostIR,
```

```

    pub report: ReportIR,
    pub analysis_time_ms: u64,
}

```

The IrPassExt blanket trait adds a run() method that chains build → compute\_metrics → validate:

```
impl<T: IrPass> IrPassExt for T {}
```

Convenience helpers in engine.rs / runner.rs: - analyze\_file(path) — reads JSON from disk, runs pipeline - analyze\_json(json\_str) — from string - validate\_json(json\_str) — parse-only, no analysis - get\_model\_summary(config) — quick model overview without full pipeline

## 8. Supporting Libraries

### neurax-formulas

12 formula modules, one per architecture family:

| Module           | Covers                                                                                                                |
|------------------|-----------------------------------------------------------------------------------------------------------------------|
| attention.rs     | Standard MHA: $4b_{sh}^2$ ; GQA: $2b_{sh}(h_{kv}/h_q + 1)$ ; FlashAttention tiled                                     |
| mlp.rs           | Dense FFN: $2b_{sh} \cdot d_{ff}$ ; Gated (SwiGLU): $3b_{sh} \cdot d_{ff}$                                            |
| conv.rs          | Conv2D: $2 \cdot C_{in} \cdot C_{out} \cdot K^2 \cdot H_{out} \cdot W_{out} \cdot B$ ; Depthwise; Grouped             |
| embedding.rs     | $vocab\_size \times embedding\_dim$ params; fwd = table lookup                                                        |
| normalization.rs | LayerNorm: $5n$ ; RMSNorm: $4n$ ; BatchNorm: $6n$                                                                     |
| moe.rs           | Router: $B \cdot S \cdot H \cdot E$ ; Expert: $top\_k \times mlp\_flops$ ; Load balancing overhead                    |
| ssm.rs           | Mamba: $O(B \cdot L \cdot D \cdot N)$ linear; S4: structured convolution; H3; RWKV                                    |
| rnn.rs           | LSTM: $8 \cdot B \cdot T \cdot H^2$ ; GRU: $6 \cdot B \cdot T \cdot H^2$ ; Vanilla RNN: $2 \cdot B \cdot T \cdot H^2$ |

| Module       | Covers                                                                                   |
|--------------|------------------------------------------------------------------------------------------|
| diffusion.rs | U-Net per timestep:<br>$\sum(\text{conv\_flops\_per\_block})$ ;<br>timestep conditioning |
| gnn.rs       | Message passing: $E \cdot D^2$ (edges<br>$\times$ feature dim <sup>2</sup> )             |
| custom.rs    | AST evaluator via evalexpr;<br>validates formula safety                                  |

Custom formula evaluation is sandboxed — only whitelisted variable names (B, S, H, D, etc.) are allowed.

### neurax-hardware-db

Static database of 20+ GPU specifications loaded at startup:

| GPU              | VRAM   | FP16 TFLOPS | FP8 TFLOPS | BW (GB/s) | NVLink |
|------------------|--------|-------------|------------|-----------|--------|
| NVIDIA H200      | 141 GB | 989         | 3958       | 4800      | □      |
| NVIDIA H100-SXM  | 80 GB  | 989         | 3958       | 3352      | □      |
| NVIDIA H100-PCIe | 80 GB  | 756         | 3026       | 2000      | □      |
| NVIDIA A100-SXM  | 80 GB  | 312         | —          | 2039      | □      |
| NVIDIA A100-PCIe | 40 GB  | 312         | —          | 1555      | □      |
| NVIDIA RTX 4090  | 24 GB  | 165         | 660        | 1008      | □      |
| NVIDIA L40S      | 48 GB  | 362         | 1448       | 864       | □      |
| NVIDIA V100      | 32 GB  | 125         | —          | 900       | □      |
| NVIDIA T4        | 16 GB  | 65          | —          | 300       | □      |
| ...and 11 more   |        |             |            |           |        |

Also includes CPU specs (AMD EPYC, Intel Xeon) and interconnects (NVLink3/4, PCIe4/5, InfiniBand-NDR).

## 9. User Interfaces

### neurax-cli

The neurax binary provides three commands:

```
# Full analysis – outputs Markdown or JSON report
neurax analyze model.json [-o report.md] [-f json|markdown]

# Parse validation only – fast schema check
```

```
neurax validate model.json
```

```
# Quick one-line summary
neurax summary model.json
```

### neurax-tui

A full **Ratatui** terminal UI (ratatui + crossterm):

| NEURAX – AI Architecture Analyzer                                            |         |                                                                                                                                            |          | v0.1.0 |
|------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| Overview                                                                     | Compute | Memory                                                                                                                                     | Hardware |        |
| [Model Selector]<br>> Llama-3-70B<br>GPT-4-Turbo<br>Mistral-7B<br>Mamba-2.8B |         | [Metrics Panel]<br>Parameters: 70.6B<br>Peak VRAM: 147.3 GB<br>FLOPs/step: 1.83e15<br>Bottleneck: memory-bound<br>Training cost: \$124,000 |          |        |
| [Comparison vs Real]<br>Our: 147GB / Ref:142G<br>+3.5% variance              |         | [Recommendations]<br>H001: Enable grad checkp.<br>H005: Consider ZeRO-3                                                                    |          |        |

**Key bindings:** ↑↓/jk model select · Enter compile · Tab/1-4 switch tabs · r refresh · q quit

## 10. Layer 4 — neurax-mlir — The Code Generation Backend

### Position in the Stack

neurax-mlir is the **second compiler stage** — it sits after neurax-core and receives computed IR metrics, then encodes them as **MLIR operations** using the melior crate (Rust bindings to LLVM's MLIR C API).

```
AnalysisResult (from neurax-core)
  |
  ▼ integration.rs
NeuraxModule (MLIR in-memory module)
  |
```

▼ passes/ (dialect lowering)  
 Standard MLIR dialects (linalg, arith, affine, gpu, nvvm...)  
 |  
 ▼ LLVM backend  
 PTX / x86 / ARM / WASM binary

## MLIR Dialects Defined

neurax-mlir defines **13 MLIR dialects** mirroring the Neurax IR:

| Dialect     | Namespace              | Key Operations                                                                                           |
|-------------|------------------------|----------------------------------------------------------------------------------------------------------|
| arch        | mlir::neurax::arch     | arch.model,<br>arch.layer,<br>arch.global_params,<br>arch.metrics,<br>arch.repeat,<br>arch.layer_pattern |
| graph       | mlir::neurax::graph    | graph.node,<br>graph.edge,<br>graph.dag                                                                  |
| tensor      | mlir::neurax::tensor   | tensor.info,<br>tensor.shape,<br>tensor.liveness                                                         |
| operator    | mlir::neurax::operator | op.matmul,<br>op.attention,<br>op.gqa, op.conv2d,<br>op.mamba, op.lstm,<br>...                           |
| compute     | mlir::neurax::compute  | compute.flops,<br>compute.com-<br>pute.complexity,<br>compute.intensity                                  |
| memory      | mlir::neurax::memory   | mem.peak_vram,<br>mem.liveness_interval,<br>mem.oom_risk                                                 |
| hardware    | mlir::neurax::hardware | hw.gpu_profile,<br>hw.roofline,<br>hw.latency                                                            |
| parallelism | mlir::neurax::par      | par.data,<br>par.tensor,<br>par.pipeline,<br>par.hybrid,<br>par.zero                                     |

| Dialect      | Namespace              | Key Operations                                         |
|--------------|------------------------|--------------------------------------------------------|
| cost         | mlir::neurax::cost     | cost.training,<br>cost.energy,<br>cost.inference       |
| report       | mlir::neurax::report   | report.final,<br>report.metric,<br>report.diagnostic   |
| training     | mlir::neurax::training | train.optimizer,<br>train.precision,<br>train.schedule |
| data         | mlir::neurax::data     | data.input,<br>data.pipeline                           |
| optimization | mlir::neurax::opt      | opt.fuse,<br>opt.tile,<br>opt.quantize                 |

### TableGen Definitions (.td files)

The include/ directory contains LLVM TableGen specifications for each dialect. For example, ArchitectureOps.td defines:

```
def ModelOp : Arch_Op<"model"> {
  let arguments = (ins StrAttr:$name, StrAttr:$model_type);
  let regions = (region AnyRegion:$layers);
}

def LayerOp : Arch_Op<"layer"> {
  let arguments = (ins StrAttr:$id, StrAttr:$layer_type,
    OptionalAttr<I64ArrayAttr>:$input_shape,
    OptionalAttr<I64ArrayAttr>:$output_shape);
}

def ArchMetricsOp : Arch_Op<"metrics"> {
  let arguments = (ins I64Attr:$total_parameters, I64Attr:$num_layers);
}
```

These .td files are the **ground-truth schema** for the MLIR layer. They can be processed by mlir-tblgen to auto-generate C++ dialect implementations.

### MLIR Passes

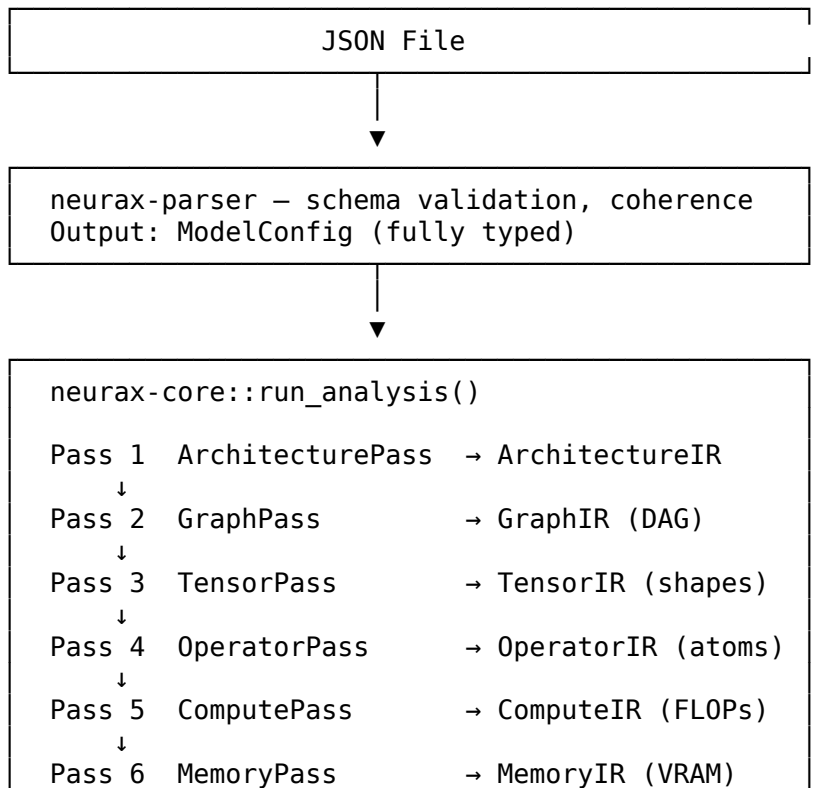
Six lowering passes defined in src/passes/:

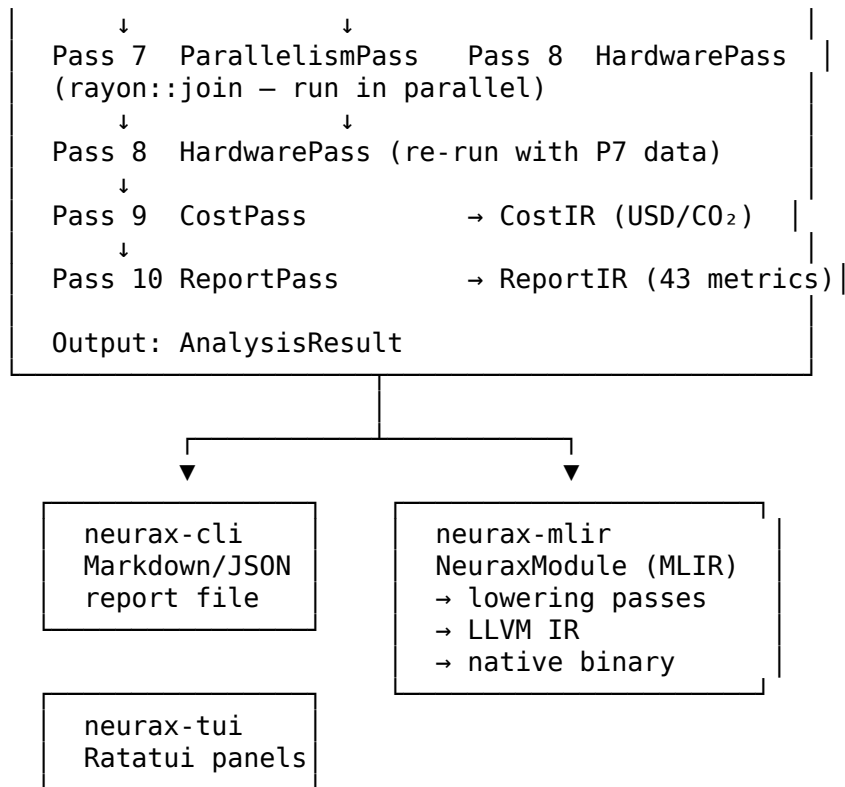
| Pass             | Transforms                                      |
|------------------|-------------------------------------------------|
| ArchitecturePass | arch.* → func.func + arith.*                    |
| ComputePass      | compute.flops → arith.mulf chains               |
| MemoryPass       | mem.* → buffer allocation ops                   |
| HardwarePass     | hw.roofline → target-specific performance hints |
| ParallelismPass  | par.* → gpu.launch + collectives                |
| CostPass         | cost.* → metadata annotations                   |

### Current State

neurax-mlir is **architecturally sound but not yet wired** to neurax-core. The integration.rs functions take raw scalar values (not AnalysisResult), so the bridge must be built. The Cargo.toml for neurax-mlir has no dependency on neurax-ir or neurax-core.

## 11. Full End-to-End Data Flow





## 12. Diagnostic System

All passes emit structured diagnostics collected in `NeuraxContext`:

### Code Classes

| Code | Severity | Meaning                                                |
|------|----------|--------------------------------------------------------|
| E001 | Error    | OOM risk — peak VRAM exceeds GPU capacity              |
| E002 | Error    | Shape gate blocked — insufficient dimension resolution |
| E003 | Error    | Custom formula evaluation failed                       |
| E004 | Error    | Unsupported layer type                                 |
| E005 | Error    | Cycle detected in computation graph                    |
| W001 | Warning  | Custom layer without formula — estimation used         |
| W002 | Warning  | Symbolic dimensions remaining in shapes                |
| W003 | Warning  | ZeRO not recommended for this config                   |
| W004 | Warning  | Flash Attention not enabled                            |

| Code | Severity | Meaning                          |
|------|----------|----------------------------------|
| W005 | Warning  | Memory close to GPU limit (>80%) |
| W006 | Warning  | Inefficient parallelism strategy |
| I001 | Info     | GQA detected                     |
| I002 | Info     | MoE detected                     |
| I003 | Info     | Flash Attention detected         |
| H001 | Hint     | Enable gradient checkpointing    |
| H002 | Hint     | Enable Flash Attention           |
| H003 | Hint     | Consider INT8 quantization       |
| H004 | Hint     | Increase micro-batches for PP    |
| H005 | Hint     | ZeRO-3 recommended               |

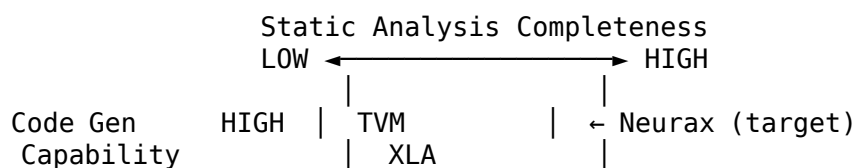
Every diagnostic includes a message, suggestion (actionable fix), layer\_id (where it occurred), and precision\_impact (0.0-1.0 — how much this degrades analysis accuracy).

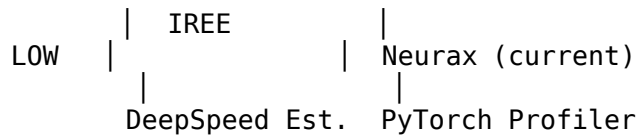
## 13. Comparisons with Peer Tools

### Feature Matrix

| Feature                                     | Neurax          | PyTorch Profiler  | DeepSpeed Estim |
|---------------------------------------------|-----------------|-------------------|-----------------|
| <b>Static analysis (no GPU needed)</b>      | ☐               | ☐ Needs runtime   | △ Partial       |
| <b>Multi-architecture support</b>           | ☐ 8 types       | △ PyTorch only    | △ Transformer   |
| <b>Memory breakdown</b>                     | ☐ 5 components  | △ Activation only | ☐               |
| <b>Parallelism recommendation</b>           | ☐ 6 strategies  | ☐                 | ☐ ZeRO only     |
| <b>Cost &amp; CO<sub>2</sub> estimation</b> | ☐               | ☐                 | ☐               |
| <b>Hardware roofline model</b>              | ☐ 4-level       | ☐                 | ☐               |
| <b>20+ GPU profiles</b>                     | ☐               | △ Measured        | ☐               |
| <b>Diagnostic system</b>                    | ☐ Structured    | △ Ad hoc          | △               |
| <b>Custom layer formulas</b>                | ☐               | ☐                 | ☐               |
| <b>Code generation</b>                      | ☐ In progress   | ☐                 | ☐               |
| <b>Multi-target compilation</b>             | ☐               | ☐                 | ☐               |
| <b>Interpretable output</b>                 | ☐ Markdown/JSON | △ Traces          | △               |

### Positioning





**Neurax’s unique value proposition:** It is the only tool that combines *full static analysis before any GPU is provisioned* with a *structured MLIR code generation pathway for any ML architecture type*.

### vs. PyTorch Profiler

PyTorch Profiler requires executing the model — you need GPU access, a loaded dataset, and a Python environment. It gives you accurate timing traces but nothing about *pre-run planning*. Neurax answers the question PyTorch Profiler cannot: “Should I even start training this configuration?”

### vs. DeepSpeed’s Memory Estimator

Excellent for ZeRO memory estimation, but only covers Transformer models and memory. No FLOPs, no hardware modeling, no cost, no code generation.

### vs. Apache TVM

TVM is a code generation compiler (input: ONNX/TensorFlow, output: optimized binary) with no analytic layer. It doesn’t tell you anything about the model’s properties before compilation. Neurax’s analytic stage would make an excellent pre-pass for TVM, informing tile sizes and parallelism strategies.

### vs. XLA / JAX

XLA is Google’s compiler for TPU/GPU optimization — JIT-based, tightly coupled to JAX/TensorFlow. Excellent for code generation, nothing for pre-run planning or multi-model comparison.

---

## 14. Quality Assessment

| Dimension                                | Score         | Notes                                                      |
|------------------------------------------|---------------|------------------------------------------------------------|
| <b>IR Architecture Design</b>            | 9.5/10        | Textbook pipeline, clean IrPass trait, proper separation   |
| <b>Model Type Coverage</b>               | 9/10          | 8 families, 50+ layer types, including SSM and GAN         |
| <b>Formula Accuracy (forward FLOPs)</b>  | 8.5/10        | Very good; GQA correct; diffusion/SSM could be deeper      |
| <b>Formula Accuracy (backward FLOPs)</b> | 6/10          | 2× forward is a rough estimate; per-op backward not done   |
| <b>Memory Modeling</b>                   | 7.5/10        | 4 components accurate; intra-op buffers missing            |
| <b>Hardware Modeling</b>                 | 8.5/10        | Industrial Roofline is sophisticated and rare              |
| <b>Parallelism Analysis</b>              | 8/10          | All 6 strategies; Amdahl/bubble ratio well-handled         |
| <b>Diagnostic System</b>                 | 8.5/10        | Production-grade structured codes and suggestions          |
| <b>Code Generation (MLIR)</b>            | 4/10          | Well-designed but disconnected; passes not yet lowering    |
| <b>Test Coverage</b>                     | 5/10          | Unit tests exist; no ground-truth benchmark fixtures       |
| <b>Overall</b>                           | <b>8.2/10</b> | Exceptional analytic engine; code-gen backend needs wiring |

## 15. Gap Analysis & Roadmap

### Critical Gaps

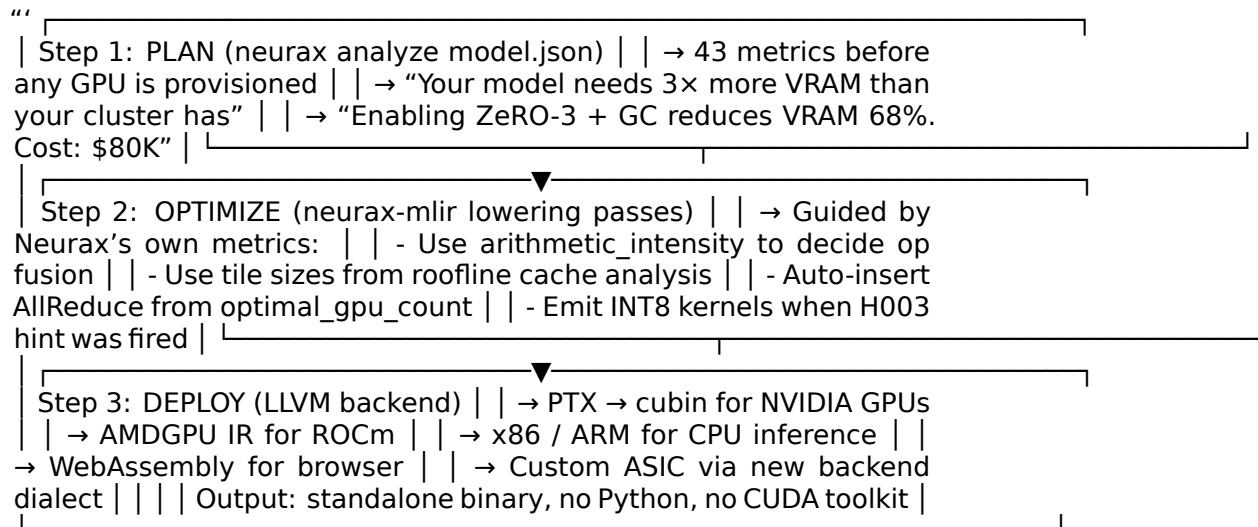
1. **neurax-mlir ↔ neurax-core bridge missing** > integration.rs takes raw scalars, not AnalysisResult. Need a From<AnalysisResult> for NeuraxModule conversion and a dependency edge in Cargo.
2. **Backward FLOPs by operator** > Currently backward = 2 × forward. Need per-op formulas (attention backward has different structure from matmul backward, especially with GQA).
3. **Intra-op buffer tracking in VRAM** > Flash Attention tiling buffers, Conv2D im2col workspace, MoE gating temp buffers. Can underestimate peak VRAM by 5-20%.
4. **Shape inference (not shape propagation)** > Currently users must provide both input\_shape and output\_shape. True compiler-style inference would derive output shapes automatically.
5. **Ground-truth test fixtures** > No benchmark suite comparing Neurax's predictions against measured GPU results. Without this, calibration cannot be validated or improved.

### Roadmap (Suggested Priority)

| Priority | Task                                                         |
|----------|--------------------------------------------------------------|
| P0       | Wire neurax-mlir to AnalysisResult — the core vision gap     |
| P1       | Per-op backward FLOPs in neurax-formulas                     |
| P1       | Intra-op buffer tracking in MemoryPass                       |
| P2       | Ground-truth fixtures in neurax-tests for top 10 open models |
| P2       | Shape inference propagation (TensorPass)                     |
| P3       | MLIR lowering passes → linalg/gpu dialects                   |
| P3       | LLVM backend emission (PTX / x86)                            |
| P4       | Multi-target compilation pipeline                            |
| P4       | Plugin/extension mechanism for custom hardware backends      |

## 16. The Full Vision

When complete, Neurax will cover the full lifecycle from model description to deployed binary:



The key architectural insight that makes Neurax different from every other compiler in this space: **the analytic pass and the code generation pass share the same IR**. The roofline ridge point that HardwareIR computes is the exact number controlling tile sizes in the MLIR lowering pass. The optimal parallelism strategy from ParallelismIR is the input to the par.hybrid MLIR op that generates collective communication. **The analysis informs the compilation** — not as heuristics, but as first-class IR attributes.

This is the vision of Neurax as a full compiler system.